

Engineering Experience

At Eden, we primarily worked in the AWS environment, and utilized a variety of languages such as TypeScript, Python, and Lua. We used DynamoDB for a no-SQL database, with our primary backend services executed via Lambda functions and EC2 instances. Our CI/CD and infrastructure was managed through AWS CDK for IaC, and a private NPM registry handled our software packages.

At Wishroute, we also primary utilized AWS, with orchestration of our key Lambda functions achieved through State Machines and Step Functions. During my internship there, my project focused on developing analytics dashboards from internal data, meaning I interacted regularly with both our no-SQL DynamoDB tables and our migrated analytics data in Postgres.

I would describe myself as a high quality coder, a description which stems from a position I hold that any code that compiles will be understood by the machine, but socially-conscious code is what will be understood by future developers. Keeping both of these audiences in mind while designing software is key, as ensuring that code will be understood by future developers is key to sustainable software development.

While at Eden, I designed and implemented a major expansion to our backend services, opting for a microservice architecture, in large part because of the business position of the organization, where the needs of our three key stakeholders meant that such an architecture created a structure which can serve the different parties successfully while maintaining efficiency and maintainability. Furthermore, encapsulating strict functionality into microservices allowed us to decouple our development, which was key for our staff makeup, with a small team of interns and an off-shore team supplementing our short-term product needs. This backend architecture needed to support two-way communication between insurance providers, healthcare providers, and patients. Each of these three external parties needed access to non-distinct subsets of our features, so designing these features using loosely coupled services allowed for flexibility as these feature subsets evolved, interactions with external partners changed, new features were developed, and as team make-up evolved. Finally, this design allowed for decoupled parallel development across timezones and evolving timelines, which was especially critical to maximize development in partnership with our offshore team.

Quality of software, and hence software development relies on symbiosis between user and system for achievement of some defined outcome. By balancing system's engineering needs with the needs, habits, and sensitivities of users, high quality software can be achieved. These engineering and user needs change based on the environment, goals, existing software paradigms, in that how software is designed or developed and how a user expects to interact with software fluctuate with different user-bases and are critical for alignment between user and software. From the perspective of the software engineer, centralizing end user experience with respect to key internal systems and architecture will allow high quality software to emerge.

Effective documentation in large software projects is a balance of abundance of information with concision. Documentation should have a clearly defined audience. This can be a specific team, or someone with a specific use for reading the documentation. For example, documentation for a developer who interacts with a software component externally (e.g. using a function from a library) needs to focus on how the component responds to external inputs, and their influence on its behavior. In other words, this documentation purpose necessitates the creation of a clear contract or expectation of behavior for the component. Alternatively, a developer that is tasked with refactoring the component has a different need that documentation can fulfill. In this context, the focus of the documentation should be on how the internal systems were designed, including the considerations, restrictions, pressures, business needs and overall circumstances of the design

at the time. At times, these audiences are at odds with one another when it comes to effective documentation. In short, I believe effective documentation should deeply consider its intended audience, and should not be written to merely check a box.

In my opinion, UX should flow. Said flow is based on the users needs, their familiarity with the software and use case, the users background, and their preconceptions of the software. As much as possible, development teams should minimize the gap between developers and users to maximize the ability for the UX to match the higher level needs of the software-user partnership. For example, software that is used by trained professionals for many hours a day should prioritize features like key-bind customization or macro support, while software that expects low familiarity should focus on highly visual interaction, where every interaction option a user has is displayed to them.

As with other aspects of software engineering, performance should also be a balance between efficiency, maintainability, utilization of current technology. In risk of repeating myself, it matters greatly how the software is being used. If an action is only taken 100 times a day, investing in a performance upgrade of 0.1s will yield less benefit than the same upgrade on an action that is taken 100k times per day. How valuable is the development investment versus the tangible outcome of a performance improvement. Additionally, performance can come at the cost of increased maintenance overhead, and advances in technology can yield performance upgrades passively.

Security should be an ensemble approach where faults are expected but not welcomed. It is prudent to always assume a security measure will be breached at some point, and to fully game out the consequences of a breach. Enough security cover should be created to minimize a cascade of breaches. By clearly outlining attack surfaces, determining how a breach in security would affect various software components and critical information or secrets, and how Cyber Security best practices and techniques can apply to a specific situation, engineers can improve their security posture and awareness.

I have limited experience with DevOps and DevSecOps, but from my experience, utilizing IaC to create automatic, repeatable deployments takes extra up front investment, but is worth the effort, especially as infrastructure increases in scale and complexity. Utilizing federated authentication services is also good because of the fact that if a single use key is compromised, a breach does not compromise future deployments. As with other aspects of software engineering, balancing granularity with coupled deployments and ensuring that feature deployments can target specific infrastructure sets will improve the sustainability of continuous development cycles.

Domain Specific Experience

My beliefs on open source development are that it should be the primary development mode in a perfect world. As more aspects of our daily lives are increasing intertwined with technology and software, the transparency and user focused philosophy naturally pushes towards sustainable, transparent software, aimed at real improvements of the users lives and direct utility. In open source projects, structuring the *meta infrastructure* is extremely important in order to leverage the community that open source software can foster. By establishing clear lines of discussion in terms of product direction; SOPs for contributions, bug reporting and fixes; as well as installation and extension documentation; the *development load* can be distributed among a community of contributors. This can lead to a degree of self-sorting, allowing people to find their niche and maximize their contributions. Of course, this opens a whole new dimension of maintenance, but when organized and communicated thoughtfully with just enough structure to maintain order and velocity, an open-source project can operate at a high level of efficiency while delivering a quality

user-focused project that is available to all. Unfortunately, I have not directly contributed to any open source projects, but in the future once I feel I have the experience, time, and alignment with a project I care about, I would very much like to make those contributions.

My knowledge of a Linux distribution is at a moderate level, having used Ubuntu exclusively for almost 16 months. In that time I have familiarized myself with the basics of interacting with the system, making personalization, and leveraging its incredible features. It is an almost weekly occurrence that I am learning of new systems, features, or strategies that Ubuntu makes possible that improve my user experience, or open up new possibilities with respect to integrations with other software systems. From my understanding, a Linux distribution consists of the operating system and then a desktop environment. The operating system interacts with the kernel and low-level systems to carry out processes and facilitate user interaction and behavior in said desktop environment. In that sense, different distros have different ways for user interaction with the computer, and offer different levels of guardrails and exposition of internal systems to users. Similarly, the modular nature of distros allows for a whole gradient of user experiences, all built on common low-level software to some extent or another. I have gained my limited knowledge of Linux distributions through various efforts to personalize my computational experience over the last 16 months, as well as troubleshooting various issues that have arisen in my system throughout that time. For example, I was experiencing an issue with my Bluetooth connection, necessitating my investigation of the various daemons and services that govern Bluetooth and manage the service for the user. After tracing the issues all the way to my Bluetooth card, I was able to determine that my motherboard was not giving any power to the card, and hence the service was not working at all.

With respect to software packaging, I have had limited exposure, but at Eden we utilized a private NPM registry for our package management and deployment. Combined with AWS CDK for IaC and GitHub Actions for CI/CD, this is how we managed our software packages. Some of our micro-services utilized Docker for their builds on various AWS services like EC2.

To engineer a release pipeline for a major software project, I would first build a simple automated pipeline that initiates on changes merged into main. This would trigger linting and automated test harnesses, create the build artifact, and then deploy to a staging environment for validation. In terms of automating this process, a CI/CD service such as GitHub Actions or a basic Jenkins setup alongside Docker would provide automated and standardized builds. Within this process, critical steps before a release would need to include extensive code review, automated test validation, and checking release notes and updating deployment documentation before promotion to production can occur.

Education

In my final year of high school for both mathematics and my home language, I ranked in the top 10 percent of my graduating class. I was in both the highest level of math and English. In the final rankings the graduating class, based on GPA, I was well within the top 20 of a class of 300 students.

My SAT score was a 1460 Superscore with a 720 in Mathematics and 740 in English.

Based on the GPA rankings in high school, I can say that I was around the top 5% of students in my year.

Outside of class, I was very involved in the soccer team, being one of the leadership captains. My responsibilities off the pitch included making sure equipment, players, and mentality were in working order to maximize our practice and meeting times. Alongside this, I also managed the discipline system, keeping track of any penalties or fines accrued by team members and making sure these debts were paid. On the

pitch, I had a key leadership role to organize our defensive structure and dynamically adapt based on game state and individual match ups to maximize our advantages.

Additionally, in my senior year, I won the Sportsmanship award for the school, based on my consistent actions throughout my athletic career, and one key instance where I acted as a de-escalator after a tough play-off loss, when a small group of our school's supporters started to verbally abuse the referee and opposing team. A coach from a rival school witnessed this scene and made sure to let my coach know and to commend me for my actions.

Outside of athletics, I was the Recording Secretary for the General Student council. In this position I met regularly with my other council members to organize school events and fundraisers.

For university, I went to Northeastern, after choosing between there, the University of Pittsburgh, and Boston University. The city, campus environment, and culture of each of these schools aligned well with my needs, but the deciding factor in favor of Northeastern was the experiential learning program. This program focused on learning opportunities outside of the classroom, including an extensive co-op or internship network, as well as a vast set of study abroad opportunities to complement the experience in Boston. I am glad to say I was able to take advantage of both of these opportunities. I worked as a Software Engineer intern at Wishroute, where I was able to gain hands-on experience at a post-revenue startup. To complement my academic experience in Boston at Northeastern, I participated in a study abroad program in Delft, the Netherlands, learning about Sustainable Transportation and Urban Planning at the University of Delft. From this experience, I was able to draw engineering parallels and immerse myself in the Dutch engineering paradigm, which centralizes three pillars to drive pragmatic and user focused design and implementation.

At Northeastern, I graduated with a Bachelor's of Computer Science, concentrating in Artificial Intelligence, as well as a minor of Spanish Language and Culture, with a GPA of 3.41 out of 4. This GPA is a relatively fair reflection of my abilities and performance throughout my time at Northeastern. While it might not be a perfect score, it reflects the intensity of the computer science program, where I was consistently challenged academically. Some of the intense and challenging courses included Theory of Computation which was the hardest academically and conceptually, while the Software Development course was extensive and rigorous in its replication of the industry, both the good and the bad aspects of it.

In my later years at Northeastern, I wanted to start looking for intersections with my newfound interest, appreciation, and knowledge of Urban Planning and Transportation Systems. More specifically, I was searching for opportunities where I can continue my own academic development with respect to computer science, while having goals and outcomes of the project that drive direct impact in the adoption of Active Transportation Networks in Boston. Through networking at various events around the city, I connected with the Boston Cyclist Union's Data Science team, and developed a project that aligns with their goals. Throughout the summer we worked together, I developed the 311 Infrastructure Issues Identification project. This project leveraged public 311 Illegal Parking Reports and used various Natural Language Processing techniques to identify different types of Active Transportation infrastructure issues such as blocked bike lanes and crosswalks, in order to create a interactive visualization of the geospatial data contained within these 311 reports. The result is a map which clearly shows the distinct pain points where the networks need to be improved, allowing funding to be used efficiently and result in real improvements to the network.

Context

While I can't speak definitively about the business strategy of Canonical, or offer suggestions for commercial positioning or risky strategies due to a lack of experience and knowledge in this context, I can definitively say that the values, commitments, and direction of Canonical align with my own convictions, morals, and philosophy. The commitment and support to open-source projects speaks to the real value that Canonical sees in open-source philosophy. In the early stages of my career, I was lucky enough to gain perspective on what aspects of my career give me the most satisfaction. Alongside the immediate satisfaction of solving a tricky problem with a clever implement, being confident that my time and effort are in service of a greater goal which serves to improve society and the world at large is a deeper and more profound source of satisfaction. So far, I have been lucky enough to work for organizations who drove meaningful impact through improving healthcare outcomes or transit access. I can confidently say that Canonical would be right alongside my previous ventures and positions as ones which drive and support positive change in the world. Being a daily user of Ubuntu and GNOME has made me appreciate how open-source development can create sustainable, transparent, customizable, and useful software systems, ones that are relied upon by millions of devices and infrastructure around the globe. The ability to contribute to the goals of Canonical as well as expand my knowledge and experience of critical software systems are key aspects of this position that entice me and stimulate my eagerness and excitement for this opportunity.